

1. Swim Times.

This is a program written using Donald Knuth’s *literate programming* paradigm (see more below). It fetches best short-course yard (SCY) and long-course meter (LCM) times for a roster of swimmers. The roster began with four family swimmers:

Stella Julianna Evans — 10 Under Girls
Kalea Rose Benavente — 13–14 Girls
Kenneth Ray Evans — 11–12 Boys
Keith Santiago Evans — 11–12 Boys

and now also covers Katie Ledecky (as age-group windows) and the full College Area Swim Team (CAST) roster listed in the **SWIMMERS** table of the main program. Each swimmer carries a unique *id* used to select her on the command line; the ids are enumerated in the software requirements specification (**swim-times-srs.tex**). Thirty-one events are reported for each swimmer. Short-course yard (SCY) events: 50, 100, 200, 500, 1000, and 1650 Freestyle; 50 and 100 Butterfly; 50 and 100 Backstroke; 50 and 100 Breaststroke; and 100 and 200 Individual Medley. Long-course meter (LCM) events: 50, 100, 200, 400, 800, and 1500 Freestyle; 50, 100, and 200 Butterfly; 50, 100, and 200 Backstroke; 50, 100, and 200 Breaststroke; and 200 and 400 Individual Medley. Data is fetched live from the USA Swimming data hub’s public REST services at times-api.usaswimming.org.

2. How it works. The data hub was formerly powered by a Sisense JAQL analytics API; it has since moved to first-party REST services, and this program follows. We make two kinds of requests:

1. A *member search* (**GetMembersForFilters**) to resolve the swimmer’s *memberId*, given a name string and a substring to match the returned full name.
2. A *best-times fetch* (**POST /BestTimes**) issued once per stroke-and-distance pair; each record delivers a stroke, distance, course, and formatted time — from which the event code (e.g. 100 FR SCY) is reassembled — together with the swim date, meet name, and the motivational standard attained (B, BB, A, AA, AAA, AAAA, or an elite label such as **Nats** or **Trials**).

All HTTP communication is handled by **libcurl**. JSON responses are scanned with simple string operations rather than a full parse tree.

3. Literate programming. Donald Knuth introduced *literate programming* in 1984 as a way of writing software that is meant to be read by human beings first and executed by computers second. Rather than annotating code with comments, a literate program interweaves prose and code in a single source document. The prose explains the *why*—the motivation, the design decisions, the mathematical reasoning—while the code expresses the *how*. The two live together in one file (conventionally given the extension **.w**) and are separated only at build time by two companion tools: **ctangle** and **cweave**.

4. ctangle. **ctangle** is the *tangling* tool. It reads a **.w** source file and extracts the C code sections, assembling them in the order dictated by named chunk references rather than the order in which they appear in the document. The result is a plain **.c** file that a standard C compiler can process without any knowledge of literate programming. In this project, running

```
ctangle swim-times.w
```

produces **swim-times.c**, which is then compiled with **cc** and linked against **libcurl** to create the **swim-times** executable. The generated **.c** file should be treated as a build artefact: the **.w** file is the true source of record.

5. cweave. `cweave` is the *weaving* tool. It reads the same `.w` source and produces a `.tex` file formatted for `pdftex` using the `cwebmac` macro package. `cweave` pretty-prints all C code with bold keywords, italic identifiers, and cross-references, and numbers every named chunk so the reader can follow the program's logical structure independently of its physical layout. An index of identifiers and a table of contents are generated automatically. Running

```
cweave swim-times.w
```

produces `swim-times.tex`; running `pdftex` on that file yields the typeset documentation you are reading now.

6. Compilation. After tangling with `ctangle`:

```
gcc -O2 -o swim-times swim-times.c $(curl-config --libs)
```

7. Program structure. The top-level arrangement of the tangled output is:

- ⟨ Includes 9 ⟩
- ⟨ Global constants 10 ⟩
- ⟨ Option flags 16 ⟩
- ⟨ Type definitions 19 ⟩
- ⟨ HTTP utilities 23 ⟩
- ⟨ JSON scanner 31 ⟩
- ⟨ Database pipe 34 ⟩
- ⟨ Person lookup 42 ⟩
- ⟨ Times fetch 50 ⟩
- ⟨ Offline fetch 67 ⟩
- ⟨ Main function 73 ⟩

8. Includes and constants.

```

9.  <Includes 9> ≡
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <curl/curl.h>
#include <libpq-fe.h>

```

This code is used in section 7.

10. `TIMES_API` is the base URL of the USA Swimming public times service that backs `data.usaswimming.org`. `EVENTS` lists all event codes we query: twelve short-course yard (SCY) events plus two additional SCY distance events (1000 FR and 1650 FR) and seventeen long-course meter (LCM) events, for a total of thirty-one.

The global constants are split into three sub-modules so that no single module exceeds twenty-four lines.

```

<Global constants 10> ≡
  <Numeric constants 11> <Data hub credentials 12> <Event table 13>

```

This code is used in section 7.

11. The two numeric limits used throughout the program.

```

<Numeric constants 11> ≡
#define NUM_EVENTS 31
#define MAX_TIMES 200

```

This code is used in section 10.

12. The base URL of the times service. Note there is deliberately no hard-coded bearer token here. The USA Swimming data hub retired the Sisense JAQL API this program originally targeted (its embedded token now returns 401 `db_unauthorized`, and the current front-end no longer exposes any Sisense credential). The replacement REST services at `times-api.usaswimming.org` authenticate anonymous callers not with a bearer token but with three custom headers — *Usas – Sub – Id: Anonymous*, *AppName: DataHub*, and a client-generated *Device – Id*. The *Device – Id* is minted at run time by `build_device_id` (see the HTTP utilities), so the “token” is obtained dynamically on every run rather than baked into the source.

```

<Data hub credentials 12> ≡
static const char TIMES_API[] = "https://times-api.usaswimming.org/swims/TimesSearch";

```

This code is used in section 10.

13. The event-code table is split into SCY and LCM sub-lists so the array initialiser fits within the line limit.

```

<Event table 13> ≡
static const char *EVENTS[NUM_EVENTS] = {<SCY events 14> <LCM events 15>};

```

This code is used in section 10.

14. The fourteen short-course yard event codes.

```

<SCY events 14> ≡
"50_FR_SCY", "100_FR_SCY", "200_FR_SCY", "500_FR_SCY", "1000_FR_SCY", "1650_FR_SCY",
"50_FL_SCY", "100_FL_SCY", "50_BK_SCY", "100_BK_SCY", "50_BR_SCY", "100_BR_SCY",
"100_IM_SCY", "200_IM_SCY" ,

```

This code is used in section 13.

15. The seventeen long-course meter event codes.

⟨ LCM events 15 ⟩ ≡

```
"50_FR_LCM", "100_FR_LCM", "200_FR_LCM", "400_FR_LCM", "800_FR_LCM", "1500_FR_LCM",
"50_FL_LCM", "100_FL_LCM", "200_FL_LCM", "50_BK_LCM", "100_BK_LCM", "200_BK_LCM",
"50_BR_LCM", "100_BR_LCM", "200_BR_LCM", "200_IM_LCM", "400_IM_LCM"
```

This code is used in section 13.

16. The program accepts two optional flags on the command line.

The `-o` flag takes a comma-separated list of one or more tokens. A token is either one of four *behaviour keywords* or a *swimmer id*:

- **fastest** — print only the single fastest time per event.
- **csv** — emit CSV lines (swimmer name on every line) instead of a table.
- **store** — write each row to the local Postgres database `swim-times` via `libpq`; combine with `csv` to also print to the terminal.
- **offline** — skip all network calls; read times from the local Postgres `swim-times` database instead.

Any other token is treated as a swimmer id (e.g. `stella`, `kalea`, `ledecky14`, `glass-layla`); see the **SWIMMERS** roster for the full list. When no swimmer id is given every swimmer is processed. Tokens may be combined, e.g. `-o stella,fastest` or `-o kalea,csv,store`. When no swimmer keyword is specified all swimmers (subject to mode) are shown.

The `-e` flag takes one or more comma-separated event codes (e.g. `-e "100 FR SCY,50 FL LCM"`) and restricts output to those events only. The flag may also be repeated (e.g. `-e "100 FR SCY" -e "50 FL LCM"`). When `-e` is omitted all thirty-one events are reported.

If no options are given at all the program prints a usage message and exits.

⟨ Option flags 16 ⟩ ≡

```
#define OPT_FASTEST (1 << 0) /* print only the single fastest time */
#define OPT_CSV (1 << 1) /* emit CSV lines instead of a table */
#define OPT_STORE (1 << 2) /* also persist rows to Postgres */
#define OPT_OFFLINE (1 << 3) /* read from Postgres, skip network */
```

See also section 17.

This code is used in section 7.

17. These are the behaviour flags (how to render or where to read/write). Swimmer *selection*—which people to fetch—is no longer encoded as one bit per swimmer, because the roster now holds several dozen entries and would overflow the flag word. Instead each swimmer carries a unique string *id* (see the *Swimmer* record) and any `-o` token that is not one of the four behaviour keywords above is treated as a swimmer id and pushed onto `g_sel`. When `g_nsel` is zero every swimmer is processed; otherwise only those whose *id* appears in `g_sel`.

⟨ Option flags 16 ⟩ +≡

```
#define MAX_SEL 64 /* max distinct swimmer ids selectable at once */
static int g_opts = 0; /* behaviour flags; see above */
static const char *g_sel[MAX_SEL]; /* selected swimmer ids (heap copies) */
static int g_nsel = 0; /* number of entries in g_sel */
static char g_events[NUM_EVENTS][32]; /* event codes requested via -e */
static int g_nevents = 0; /* number of entries in g_events */
```

18. Data structures.

19. A *Buffer* holds a dynamically-grown heap string. `libcurl` appends each response chunk to it via the write callback.

⟨Type definitions 19⟩ ≡

```
typedef struct {
    char *data;
    size_t size;
} Buffer;
```

See also sections 20 and 21.

This code is used in section 7.

20. A *TimeRow* records one swim result. *sort_key* is the swim time in seconds (smaller is faster), computed by *time_to_seconds*; *time* is the formatted string (e.g. 1:02.45); *date* is the swim date formatted as YYYY-MM-DD; *standard* is the motivational standard attained (e.g. B, BB, A); and *meet* is the meet name. The `BestTimes` feed populates every field; only when a swim earned no motivational standard is *standard* left empty.

⟨Type definitions 19⟩ +≡

```
typedef struct {
    double sort_key;
    char time[32];
    char date[16];
    char standard[48];
    char meet[256];
} TimeRow;
```

21. A *Swimmer* record holds the per-swimmer search parameters: a query string sent to the person-search API, a lower-case substring used to identify the correct row, a flag bit used to filter output, and an optional age window expressed as ISO YYYY-MM-DD dates. The typedef lives in this section (rather than next to the `SWIMMERS` array in `main`) so that *offline_fetch* can reference it without a forward declaration.

⟨Type definitions 19⟩ +≡

```
typedef struct {
    const char *id; /* unique selection id, e.g. "stella" */
    const char *search_query; /* Name string to search for */
    const char *match_substr; /* Lower-case substring to match */
    const char *date_min; /* NULL or "YYYY-MM-DD" inclusive lower */
    const char *date_max; /* NULL or "YYYY-MM-DD" inclusive upper */
} Swimmer;
```

22. HTTP utilities.

23. The `libcurl` write callback appends each incoming chunk to a **Buffer**, growing the allocation as needed. It maintains a null terminator so the buffer can always be treated as a C string.

⟨HTTP utilities 23⟩ ≡

```
static size_t write_cb(void *ptr, size_t size, size_t nmemb, void *ud)
{
    Buffer *b = (Buffer *) ud;
    size_t n = size * nmemb;
    char *p = realloc(b->data, b->size + n + 1);
    if (!p) return 0;
    b->data = p;
    memcpy(b->data + b->size, ptr, n);
    b->size += n;
    b->data[b->size] = '\0';
    return n;
}
```

See also sections 24, 25, and 26.

This code is used in section 7.

24. `base64` encodes a null-terminated string into standard base64. It is used only to shape the *Device – Id* header, so a compact implementation that never overflows `out` is all that is required.

⟨HTTP utilities 23⟩ +=

```
static const char B64[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456\
789+/";

static void base64(const char *in, char *out, size_t out_sz)
{
    size_t len = strlen(in), i = 0, o = 0;
    while (i + 3 ≤ len ∧ o + 4 < out_sz) {
        unsigned t = ((unsigned char) in[i] << 16) | ((unsigned char) in[i + 1] << 8) | (unsigned char)
            in[i + 2];
        out[o++] = B64[(t >> 18) & 63];
        out[o++] = B64[(t >> 12) & 63];
        out[o++] = B64[(t >> 6) & 63];
        out[o++] = B64[t & 63];
        i += 3;
    }
    if (len > i ∧ o + 4 < out_sz) {
        size_t rem = len - i;
        unsigned t = (unsigned char) in[i] << 16;
        if (rem ≡ 2) t |= (unsigned char) in[i + 1] << 8;
        out[o++] = B64[(t >> 18) & 63];
        out[o++] = B64[(t >> 12) & 63];
        out[o++] = rem ≡ 2 ? B64[(t >> 6) & 63] : '=';
        out[o++] = '=';
    }
    out[o] = '\0';
}
```

25. *device_id* returns the cached *Device – Id* value the times service requires, building it once on first use. We reproduce the shape the data hub’s front-end produces: base64 of "platform – vendor – fingerprint – millis", then the first five characters repeated after the fifteenth. The server validates only the format, so a per-run fingerprint (process id plus clock) is sufficient.

```
< HTTP utilities 23 > +=
static const char *device_id(void)
{
    static char id[512];
    if (id[0]) return id;
    char raw[128], n[256];
    long now = (long) time(Λ);
    snprintf(raw, sizeof raw, "Linux_ _curlswim_ _%lx%lx_ _%ld000", (unsigned long)
        getpid(), (unsigned long) now, now);
    base64(raw, n, sizeof n);
    snprintf(id, sizeof id, "%.15s%.5s%s", n, n, n + 15);
    return id;
}
```

26. *http_request* performs one request — a GET when *body* is Λ , otherwise a POST carrying *body* — and returns the response as a null-terminated heap string, or Λ on failure. The caller frees the result. Every request carries the anonymous data-hub headers instead of a bearer token.

```
< HTTP utilities 23 > +=
static char *http_request(const char *url, const char *body){< Initialize curl handle 27 >< Issue HTTP
    request and return 28 >>
```

27. The easy handle is created, the anonymous auth headers are assembled, and all curl options are configured before the request is issued.

```
< Initialize curl handle 27 > =
CURL *curl = curl_easy_init();
if (!curl) return Λ;
Buffer buf = {Λ, 0};
char dev_hdr[560];
snprintf(dev_hdr, sizeof dev_hdr, "Device-Id: _%s", device_id());
struct curl_slist *hdrs = Λ;
hdrs = curl_slist_append(hdrs, "Content-Type: _application/json");
hdrs = curl_slist_append(hdrs, "AppName: _DataHub");
hdrs = curl_slist_append(hdrs, "Usas-Sub-Id: _Anonymous");
hdrs = curl_slist_append(hdrs, dev_hdr);
curl_easy_setopt(curl, CURLOPT_URL, url);
curl_easy_setopt(curl, CURLOPT_HTTPHEADER, hdrs);
if (body) curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body);
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);
curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
```

This code is used in section 26.

28. The request is executed; on success the accumulated buffer is returned to the caller; on failure the partial buffer is freed and Λ is returned.

```
 $\langle$  Issue HTTP request and return 28  $\rangle \equiv$   
    CURLcode rc = curl_easy_perform(curl);  
    curl_slist_free_all(hdrs);  
    curl_easy_cleanup(curl);  
    if (rc  $\neq$  CURLE_OK) {  
        free(buf.data);  
        return  $\Lambda$ ;  
    }  
    return buf.data;
```

This code is used in section 26.

29. JSON scanner.

30. The REST responses are JSON arrays of objects with named fields. A member-search row looks like

```
"memberId":"6CD35348E5824C","fullName":"Katie Genevieve Ledecy",...
```

and a best-times row like

```
"strokeAbbreviationd":"FR","distance":50,"courseCode":"SCY","swimTime":"22.64r"
```

Rather than build a full parse tree we scan forward for a named key and extract the immediately following value, advancing a position pointer. The scanner is format-agnostic: it served the old Sisense "text"/"data" cells and serves these named fields unchanged.

31. *scan_string* finds the first "key" at or after **pos*, copies the string value that follows into *out* (at most *max_len* − 1 bytes), and advances **pos* past it. Returns 1 on success, 0 if the key is absent.

(JSON scanner 31) ≡

```
static int scan_string(const char *key, const char **pos, char *out, size_t max_len)
{
    char needle[128];
    snprintf(needle, sizeof needle, "\"%s\"", key);
    const char *p = strstr(*pos, needle);
    if (!p) return 0;
    p += strlen(needle);
    while (*p == ' ' || *p == ':') p++;
    if (*p != '"') return 0;
    p++;
    const char *e = strchr(p, '"');
    if (!e) return 0;
    size_t len = (size_t)(e - p);
    if (len >= max_len) len = max_len - 1;
    memcpy(out, p, len);
    out[len] = '\0';
    *pos = e + 1;
    return 1;
}
```

This code is used in section 7.

32. Numeric fields in these feeds (*distance*, *ages*, *ids*) are not needed by the scanner: every value the program consumes — meet name, event code, date, time, and standard — arrives as a JSON string, so a single string extractor suffices.

33. Database connection.

34. The **store** option streams rows into the local Postgres database **swim-times**; the **offline** option reads previously-stored rows back. Both reach Postgres *programmatically* via the standard client library **libpq** — no **psql** child process or wrapper script is involved (this is requirement #3 of **requirements4.txt**: “Access the Postgres database programmatically, not through scripts”). A connection string can be supplied via the **SWIM_TIMES_PGCONNINFO** environment variable; if unset the program connects to a local container with the keyword string **dbname=swim-times user=postgres host=localhost port=5432**. Per-row inserts use **INSERT ... ON CONFLICT ... DO NOTHING** keyed on the unique tuple (**swimmer**, **event**, **swim_time**, **swim_date**, **meet**) so a duplicate silently no-ops without aborting the surrounding work (requirement #4: “check for duplicates before adding ... and recover appropriately”).

```
<Database pipe 34> ≡
  <Resolve connection string 35><Open and close DB connection 36><Insert one row 40>
```

This code is used in section 7.

35. **db_conn_string** returns the libpq connection string. Override at runtime with **SWIM_TIMES_PGCONNINFO**.

```
<Resolve connection string 35> ≡
static const char *db_conn_string(void)
{
    const char *e = getenv("SWIM_TIMES_PGCONNINFO");
    return (e & *e) ? e : "dbname=swim-times user=postgres host=localhost port=5432";
}
```

This code is used in section 34.

36. **db_open** opens a libpq connection and prepares the parameterised **INSERT ... ON CONFLICT DO NOTHING** statement that all **store** writes use. **db_close** releases the connection. Both are idempotent. The **ON CONFLICT** clause keys on the same unique constraint declared by **schema.sql**, so a duplicate row is a no-op rather than an error.

```
<Open and close DB connection 36> ≡
static PGconn*db_conn = Λ;
#define DB_INSERT_STMT "swim_times_insert_v1"
<Open DB connection 37><Close DB connection 38>
```

This code is used in section 34.

37. Connection establishment. A failure is non-fatal here: the caller chooses whether to abort. In **store** mode the program continues without persisting; in **offline** mode the caller bails out.

```
<Open DB connection 37> ≡
static int db_open(void)
{
    db_conn = PQconnectdb(db_conn_string());
    if (PQstatus(db_conn) ≠ CONNECTION_OK) {
        fprintf(stderr, "Warning: libpq connection failed: %s", PQerrorMessage(db_conn));
        PQfinish(db_conn);
        db_conn = Λ;
        return 0;
    }
    <Prepare insert statement 39>return 1;
}
```

This code is used in section 36.

38. Tear-down.

⟨Close DB connection 38⟩ ≡

```
static void db_close(void)
{
    if (db_conn) {
        PQfinish(db_conn);
        db_conn = Λ;
    }
}
```

This code is used in section 36.

39. The prepared insert. Date and float casts let us pass every parameter as a text string; libpq does the parsing.

⟨Prepare insert statement 39⟩ ≡

```
PGresult *r = PQprepare(db_conn, DB_INSERT_STMT,
    "INSERT INTO swim_times (" swimmer,event,swim_time,swim_date,stan\
    dard,meet,sort_key) "VALUES ($1,$2,$3,$4::date,$5,$6,$7::flo\
    at8) "ON CONFLICT (swimmer,event,swim_time,swim_date,meet) "DO NOTHING", 7, Λ);
if (PQresultStatus(r) ≠ PGRES_COMMAND_OK) {
    fprintf(stderr, "Warning: prepare failed: %s", PQerrorMessage(db_conn));
    PQclear(r);
    PQfinish(db_conn);
    db_conn = Λ;
    return 0;
}
PQclear(r);
```

This code is used in section 37.

40. `db_insert_row` executes the prepared statement once for one `TimeRow`. A duplicate hits the `ON CONFLICT DO NOTHING` branch and is silently absorbed; any other failure logs a warning but does not abort the run. An empty `date`, `standard`, or `meet` is passed as a SQL `Λ` (a `Λ` pointer in the value array) so the `$4::date` cast does not choke on an empty string; the best-times feed leaves these fields empty.

⟨Insert one row 40⟩ ≡

```
static void db_insert_row(const char *swimmer, const char *event, const TimeRow *r)
{
    if (¬db_conn) return;
    char sk[64];
    snprintf(sk, sizeof sk, "%g", r→sort_key);
    const char *vals[7] = {swimmer, event, r→time, r→date[0] ? r→date : Λ, r→standard[0] ? r→standard : Λ,
        r→meet[0] ? r→meet : Λ, sk};
    PGresult *res = PQexecPrepared(db_conn, DB_INSERT_STMT, 7, vals, Λ, Λ, 0);
    if (PQresultStatus(res) ≠ PGRES_COMMAND_OK)
        fprintf(stderr, "Warning: insert failed: %s", PQerrorMessage(db_conn));
    PQclear(res);
}
```

This code is used in section 34.

41. Person lookup. *lookup_member_id* posts a member search for *search_query* and scans the result records for one whose full name contains *match_substr* (after lower-casing). It returns the matching swimmer's *memberId* as a heap string (an alphanumeric token such as 6CD35348E5824C, *not* the old numeric PersonKey) and, optionally, the full name in **out_name*. Returns Λ on failure. The function body is split into five sub-modules.

42.

```

⟨ Person lookup 42 ⟩ ≡
    static char *lookup_member_id(const char *search_query, const char *match_substr, const char
        **out_name){⟨ Build person-search URL 43 ⟩⟨ Build person-search body 44 ⟩⟨ Issue person-search
        request 45 ⟩⟨ Scan person-search result 46 ⟩⟨ Return person-search result 48 ⟩}

```

This code is used in section 7.

43. The URL addresses the member-search endpoint.

```

⟨ Build person-search URL 43 ⟩ ≡
    char url[512];
    snprintf(url, sizeof url, "%s/GetMembersForFilters", TIMES_API);

```

This code is used in section 42.

44. The request body is a *SFPersonSearchFilters* object; only the free-text *name* field is needed. The service matches *name* against member full names, so a distinctive surname keeps the result set small.

```

⟨ Build person-search body 44 ⟩ ≡
    char body[512];
    snprintf(body, sizeof body, "{\"name\":\"%s\"}", search_query);

```

This code is used in section 42.

45. The HTTP POST is issued; a Λ response is a fatal error.

```

⟨ Issue person-search request 45 ⟩ ≡
    char *resp = http_request(url, body);
    if (!resp) {
        fputs("Error: person lookup request failed\n", stderr);
        return  $\Lambda$ ;
    }

```

This code is used in section 42.

46. The response is a JSON array of member objects. Records are scanned until one whose lower-cased full name contains *match_substr* is found; its *memberId* is duplicated. The loop body is a separate sub-chunk so the outer scanner stays inside the twenty-four line limit.

```

⟨ Scan person-search result 46 ⟩ ≡
    char *key =  $\Lambda$ ;
    char *name =  $\Lambda$ ;
    const char *p = resp;
    while (p) {⟨ Match one person row 47 ⟩}

```

This code is used in section 42.

47. Within each record *memberId* precedes *fullName*, so we read the id first, then the name; if the name matches we keep the id already in hand.

⟨ Match one person row 47 ⟩ ≡

```

char member[64];
if ( $\neg$ scan_string("memberId",&p,member,sizeof member)) break;
char full_name[256];
if ( $\neg$ scan_string("fullName",&p,full_name,sizeof full_name)) break;
char lower[256];
size_t flen = strlen(full_name);
for (size_t i = 0; i ≤ flen; i++)
    lower[i] = (char)(full_name[i] ≥ 'A' ∧ full_name[i] ≤ 'Z' ? full_name[i] + 32 : full_name[i]);
if (strstr(lower,match_substr)) {
    key = strdup(member);
    name = strdup(full_name);
    break;
}

```

This code is used in section 46.

48. The response buffer is freed, a diagnostic is printed on failure, and the *memberId* (or Λ) is returned.

⟨ Return person-search result 48 ⟩ ≡

```

free(resp);
if ( $\neg$ key) fprintf(stderr,"Error:␣swimmer␣\"%s\"␣not␣found\n",search_query);
if (out_name) *out_name = name;
else free(name);
return key;

```

This code is used in section 42.

49. Times fetch.

50. *insertion_sort* sorts n **TimeRow** records in-place by *sort_key* ascending (smallest = fastest time). A swimmer's career rarely exceeds a few dozen entries per event, so $O(n^2)$ is entirely adequate.

```

⟨Times fetch 50⟩ ≡
static void insertion_sort(TimeRow *rows, int n)
{
    for (int i = 1; i < n; i++) {
        TimeRow tmp = rows[i];
        int j = i - 1;
        while (j ≥ 0 ∧ rows[j].sort_key > tmp.sort_key) {
            rows[j + 1] = rows[j];
            j--;
        }
        rows[j + 1] = tmp;
    }
}

```

See also sections 51, 53, 54, 55, 56, 58, and 60.

This code is used in section 7.

51. *time_to_seconds* converts a formatted swim time ("22.64r", "1:40.36", "14:59.62") to a **double** number of seconds, used as the sort key. Colon-separated groups are accumulated sexagesimally; any trailing flag character (such as the **r** marking a relay lead-off) is ignored.

```

⟨Times fetch 50⟩ +≡
static double time_to_seconds(const char *t)
{
    double total = 0;
    char buf[32];
    size_t bi = 0;
    for (const char *s = t; ; s++) {
        if (*s == ':' ∨ *s == '\0') {
            buf[bi] = '\0';
            total = total * 60 + strtod(buf, &);
            bi = 0;
            if (*s == '\0') return total;
        }
        else if (((*s ≥ '0' ∧ *s ≤ '9') ∨ *s == '.') ∧ bi < sizeof buf - 1) {
            buf[bi++] = *s;
        }
    }
}

```

52. Retrieving date, meet, and standard. The lightweight **GetBestTimesForMember** feed this program originally used returned only a stroke, distance, course, and formatted time — no swim date, meet name, or motivational standard. Those three fields are, however, reachable anonymously through the richer **POST /BestTimes** query, the same call the data-hub front-end issues to render a swimmer's "Best Times" page. Given a *memberId*, a stroke abbreviation, and a distance, **BestTimes** returns the swimmer's lifetime-best time in each course (SCY and LCM) for that stroke-and-distance pair, and each record now carries *meetName*, *swimDate*, *eventCode*, *swimTime*, and *timeStandard* — the B, BB, A, AA, AAA, AAAA, or elite (Nats, Trials, ...) level attained. This is requirement #1 and #2 of **requirements6.txt**.

53. Because **BestTimes** keys on a single stroke-and-distance pair, one POST is issued per distinct pair among the requested events (about twenty-two calls for the full thirty-one-event roster, since each call delivers both courses). Every parsed record is stored in a per-member cache *g_cache*; each per-event query is then answered from that cache without further network traffic. The *date_min* and *date_max* age-window parameters are retained for signature compatibility but are not applied online.

```

⟨Times fetch 50⟩ +≡
    typedef struct {
        char event[32];
        TimeRow row;
    } CacheEntry;
    static char g_cache_member[64] = ""; /* memberId the cache holds */
    static CacheEntry g_cache[NUM_EVENTS]; /* one best row per event */
    static int g_cache_n = 0; /* live entries in g_cache */

```

54. *normalize_date* converts the API's "Mon DD, YYYY" date (e.g. "Jan 19, 2019") to ISO YYYY-MM-DD. The ISO form sorts lexically, contains no comma (so it needs no CSV quoting), and matches the format the offline read path reconstructs with *to_char*. An unrecognised input is copied through unchanged.

```

⟨Times fetch 50⟩ +≡
    static void normalize_date(const char *in, char *out, size_t n)
    {
        static const char *M = "JanFebMarAprMayJunJulAugSepOctNovDec";
        char mon[4];
        int day = 0, yr = 0;
        if (sscanf(in, "%3s %d, %d", mon, &day, &yr) ≠ 3) {
            snprintf(out, n, "%s", in);
            return;
        }
        const char *pos = strstr(M, mon);
        int m = pos ? (int)((pos - M)/3) + 1 : 0;
        if (yr < 0 ∨ yr > 9999) yr = 0; /* bound so ISO output fits 16 bytes */
        if (m < 0 ∨ m > 99) m = 0;
        if (day < 0 ∨ day > 99) day = 0;
        snprintf(out, n, "%04d-%02d-%02d", yr, m, day);
    }

```

55. *cache_add* appends one parsed **BestTimes** record to *g_cache*, computing the numeric sort key from the formatted time and normalising the date to ISO form.

```

⟨Times fetch 50⟩ +≡
    static void cache_add(const char *event, const char *time, const char *date, const char
        *standard, const char *meet)
    {
        if (g_cache_n ≥ NUM_EVENTS) return;
        CacheEntry *e = &g_cache[g_cache_n++];
        snprintf(e-event, sizeof e-event, "%s", event);
        e-row.sort_key = time_to_seconds(time);
        snprintf(e-row.time, sizeof e-row.time, "%s", time);
        normalize_date(date, e-row.date, sizeof e-row.date);
        snprintf(e-row.standard, sizeof e-row.standard, "%s", standard);
        snprintf(e-row.meet, sizeof e-row.meet, "%s", meet);
    }

```

56. *fetch_pair* POSTs one `BestTimes` query for a stroke abbreviation and distance and parses every returned record into the cache.

⟨Times fetch 50⟩ +≡

```
static void fetch_pair(const char *member_id, const char *stroke, long dist)
{
    char url[256], body[256];
    snprintf(url, sizeof url, "%s/BestTimes", TIMES_API);
    snprintf(body, sizeof body,
        "{\"memberId\":\"%s\", \"strokeAbbreviation\":\"%s\", \"distance\":%ld}",
        member_id, stroke, dist);
    char *resp = http_request(url, body);
    if (¬resp) return;
    ⟨Parse BestTimes records 57⟩free(resp);
}
```

57. Within each record the fields appear in the fixed order *meetName*, *eventCode*, *swimDate*, *swimTime*, *timeStandard*, so the scanner reads them in that sequence, advancing its position pointer past each. *timeStandard* may be JSON *null* (no standard earned); the scan then leaves *std* empty.

⟨Parse BestTimes records 57⟩ ≡

```
const char *p = resp;
char meet[256], event[32], date[16], stime[32], std[48];
while (scan_string("meetName", &p, meet, sizeof meet)) {
    if (¬scan_string("eventCode", &p, event, sizeof event)) break;
    if (¬scan_string("swimDate", &p, date, sizeof date)) break;
    if (¬scan_string("swimTime", &p, stime, sizeof stime)) break;
    std[0] = '\0';
    scan_string("timeStandard", &p, std, sizeof std);
    cache_add(event, stime, date, std, meet);
}
```

This code is used in section 56.

58. *build_member_cache* fills *g_cache* for *member_id* by issuing one **BestTimes** POST per distinct stroke-and-distance pair drawn from the effective event list (the **-e** selection when given, otherwise all thirty-one **EVENTS**). Pairs already fetched are skipped so each POST runs at most once.

⟨Times fetch 50⟩ +≡

```
static void build_member_cache(const char *member_id)
{
    g_cache_n = 0;
    int use_e = (g_nevents > 0);
    int n = use_e ? g_nevents : NUM_EVENTS;
    char done[NUM_EVENTS][16];
    int nd = 0;
    for (int i = 0; i < n; i++) {
        const char *code = use_e ? g_events[i] : EVENTS[i];
        long dist;
        char stroke[8], key[16];
        if (sscanf(code, "%ld_7s", &dist, stroke) != 2) continue;
        snprintf(key, sizeof key, "%s_7ld", stroke, dist);
        ⟨Skip pair if already fetched 59⟩ fetch_pair(member_id, stroke, dist);
    }
    snprintf(g_cache_member, sizeof g_cache_member, "%s", member_id);
}
```

59. A linear scan of the *done* list suppresses a repeat POST for a stroke-and-distance pair whose SCY and LCM events both appear in the event list.

⟨Skip pair if already fetched 59⟩ ≡

```
int seen = 0;
for (int j = 0; j < nd; j++)
    if (strcmp(done[j], key) == 0) {
        seen = 1;
        break;
    }
if (seen) continue;
if (nd < NUM_EVENTS) snprintf(done[nd++], sizeof done[0], "%s", key);
```

This code is used in section 58.

60. *fetch_times* reports *member_id*'s best time in *event_code*. The per-member cache is rebuilt whenever the requested member changes; the matching row (if any) is then copied out and handed to the shared sort/emit chunk.

⟨Times fetch 50⟩ +≡

```
static void fetch_times(const char *member_id, const char *event_code, const char
                        *swimmer_name, int opts, const char *date_min, const char *date_max)
{
    (void) date_min;
    (void) date_max;
    if (strcmp(g_cache_member, member_id) != 0) build_member_cache(member_id);
    ⟨Select cached time for event 61⟩ ⟨Sort and emit rows 62⟩
}
```

61. The cache holds at most one best row per event, so the selection is a simple linear scan for the matching *event_code*.

```

⟨Select cached time for event 61⟩ ≡
    TimeRow rows[MAX_TIMES];
    int nrows = 0;
    for (int i = 0; i < g_cache_n ∧ nrows < MAX_TIMES; i++)
        if (strcmp(g_cache[i].event, event_code) == 0) rows[nrows++] = g_cache[i].row;

```

This code is used in section 60.

62. The sort runs once and then up to three emitters are dispatched according to the option mask: CSV to stdout, DB pipe write, or the table to stdout. CSV and DB emit can fire together; **store** without **csv** writes silently to the database while still printing the table.

```

⟨Sort and emit rows 62⟩ ≡
    insertion_sort(rows, nrows);
    int lim = (opts & OPT_FASTEST) ? (nrows > 0 ? 1 : 0) : nrows;
    if (opts & OPT_CSV) {⟨Emit rows as CSV 63⟩}
    if ((opts & OPT_STORE) ∧ db_conn) {⟨Emit rows to DB 64⟩}
    if (¬(opts & OPT_CSV)) {⟨Emit rows as table 65⟩}

```

This code is used in sections 60 and 68.

63. The CSV form on standard output is the same six-column layout honoured by the existing CAST report pipeline.

```

⟨Emit rows as CSV 63⟩ ≡
    for (int i = 0; i < lim; i++)
        printf("%s\\", "%s\\", "%s\\", "%s\\", "%s\\", "%s\\", swimmer_name ? swimmer_name : "",
            event_code, rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);

```

This code is used in section 62.

64. The DB form delegates each row to *db_insert_row*. Per-row inserts let **ON CONFLICT DO NOTHING** swallow duplicates one at a time without aborting the surrounding work — the bulk **COPY** path would have rolled back the whole batch on the first collision.

```

⟨Emit rows to DB 64⟩ ≡
    for (int i = 0; i < lim; i++) db_insert_row(swimmer_name ? swimmer_name : "", event_code, &rows[i]);

```

This code is used in section 62.

65. The table form is the human-readable per-event block printed when **csv** is not requested.

```

⟨Emit rows as table 65⟩ ≡
    printf("%s_--_%s:\n", event_code,
        (opts & OPT_FASTEST) ? "fastest_time" : "all_times_(fastest_first)");
    printf("%-12s_%-10s_%-13s%s\n", "Time", "Date", "Standard", "Meet");
    printf("%-12s_%-10s_%-13s%s\n", "-----", "-----", "-----", "----");
    for (int i = 0; i < lim; i++)
        printf("%-12s_%-10s_%-13s%s\n", rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);
    if (nrows == 0) printf("(no_times_found)\n");
    putchar('\n');

```

This code is used in section 62.

66. Offline fetch.

67. *offline_fetch* is the read counterpart to the insert path: it issues a parameterised **SELECT** via `libpq` on the shared *db_conn* and feeds each tuple into the same **TimeRow** array consumed by the shared sort/emit chunk. No network is touched and no child process is spawned. The function takes a **Swimmer** pointer so it can use both *match_substr* (for an **ILIKE** clause that maps the user keyword to whichever full name the database happens to hold) and the optional age window (*date_min*, *date_max*).

⟨ Offline fetch 67 ⟩ ≡
 ⟨ Define *offline_fetch* 68 ⟩

This code is used in section 7.

68. The fetch function is decomposed into three sub-chunks so each stays inside the twenty-four line limit.

⟨ Define *offline_fetch* 68 ⟩ ≡
static void *offline_fetch*(**const** **Swimmer** **sw*, **const** **char** **event_code*, **int** *opts*){⟨ Build offline query
 params 69 ⟩⟨ Read offline rows from *libpq* 70 ⟩⟨ Sort and emit rows 62 ⟩}

This code is used in section 67.

69. The query uses two positional parameters: the **ILIKE** pattern (*%match_substr%*) and the event code. *libpq* quotes them safely so no shell-quoting hazards apply.

⟨ Build offline query params 69 ⟩ ≡
char *like_pat*[256];
snprintf(*like_pat*, **sizeof** *like_pat*, "%s", *sw*→*match_substr*);
const **char** **params*[2] = {*like_pat*, *event_code*};
const **char** **q* = "SELECT swimmer, swim_time, "to_char(swim_date, '\n
 YYYY-MM-DD'), "standard, meet, sort_key FROM swim_times" "WHERE swimmer ILIKE\n
 \$1 AND event = \$2" "ORDER BY sort_key";

This code is used in section 68.

70. The query is executed, the result is iterated, and each tuple is copied into a **TimeRow**. The age window is honoured here so that an offline query that ignores the filter (e.g. a wider DB range) still produces age-appropriate output.

⟨ Read offline rows from *libpq* 70 ⟩ ≡
char *swimmer_buf*[256] = "";
const **char** **swimmer_name* = *swimmer_buf*;
TimeRow *rows*[**MAX_TIMES**];
int *nrows* = 0;
if (¬*db_conn*) {
fputs("Error: no DB connection for offline read\n", *stderr*);
return;
 }
PGresult **res* = *PQexecParams*(*db_conn*, *q*, 2, *Λ*, *params*, *Λ*, *Λ*, 0);
if (*PQresultStatus*(*res*) ≠ *PGRES_TUPLES_OK*) {
fprintf(*stderr*, "Error: offline query failed: %s", *PQerrorMessage*(*db_conn*));
PQclear(*res*);
return;
 }
int *total* = *PQntuples*(*res*);
for (**int** *i* = 0; *i* < *total* ∧ *nrows* < **MAX_TIMES**; *i*++) {⟨ Copy one *libpq* row 71 ⟩
PQclear(*res*);

This code is used in section 68.

71. Each tuple carries six columns matching the **SELECT** list: swimmer name, time, date, standard, meet, sort key. Rows that fall outside the swimmer's optional age window are skipped without incrementing *nrows*.

(Copy one libpq row 71) $\equiv$

```

const char *f0 = PQgetvalue(res, i, 0);
const char *f1 = PQgetvalue(res, i, 1);
const char *f2 = PQgetvalue(res, i, 2);
const char *f3 = PQgetvalue(res, i, 3);
const char *f4 = PQgetvalue(res, i, 4);
const char *f5 = PQgetvalue(res, i, 5);

if (sw-date_min  $\wedge$  strcmp(f2, sw-date_min) < 0) continue;
if (sw-date_max  $\wedge$  strcmp(f2, sw-date_max) > 0) continue;
if ( $\neg$ *swimmer_buf) {
    strncpy(swimmer_buf, f0, sizeof swimmer_buf - 1);
    swimmer_buf[sizeof swimmer_buf - 1] = '\0';
}
strncpy(rows[nrows].time, f1, sizeof rows[nrows].time - 1);
rows[nrows].time[sizeof rows[nrows].time - 1] = '\0';
strncpy(rows[nrows].date, f2, sizeof rows[nrows].date - 1);
rows[nrows].date[sizeof rows[nrows].date - 1] = '\0';
strncpy(rows[nrows].standard, f3, sizeof rows[nrows].standard - 1);
rows[nrows].standard[sizeof rows[nrows].standard - 1] = '\0';
strncpy(rows[nrows].meet, f4, sizeof rows[nrows].meet - 1);
rows[nrows].meet[sizeof rows[nrows].meet - 1] = '\0';
rows[nrows].sort_key = strtod(f5,  $\Lambda$ );
nrows++;

```

This code is used in section 70.

72. Main program.

73. We define a small **Swimmer** record to hold the person-search parameters for each swimmer. Each entry supplies a unique *id* string (the token the user passes to `-o` to select her), a *search_query* string sent to the database (ideally distinctive enough to return a small set), and a *match_substr* (lower-cased) used to identify the correct row among the results. The optional *date_min* and *date_max* fields restrict output to swims whose date falls in $[date_min, date_max]$ (inclusive, ISO YYYY-MM-DD). These are used to express age-group windows—for example Katie Ledecky’s 9- and 10-year-old swims fall between her ninth and eleventh birthdays. Kalea’s entry searches “Benavente” because the simple two-word query “Kalea Benavente” returns no results; the API requires an exact substring match against the registered name “Kalea Rose Benavente”. Kenneth’s entry uses “kenneth ray” and Keith’s uses “keith santiago” to avoid false matches on the common surname “Evans”. Katie Ledecky was born 17 March 1997, so the window 2006-03-17 through 2008-03-16 captures every swim from her ninth birthday up to (but not including) her eleventh. The same construction yields 2008-03-17 through 2010-03-16 for the 11- and 12-year-old window (`ledecky12`) and 2010-03-17 through 2012-03-16 for the 13- and 14-year-old window (`ledecky14`).

(Main function 73) $\equiv$

```
static const Swimmer SWIMMERS[] = {{"stella", "Julianna_Evans", "stella", \Lambda, \Lambda, {"kalea",
    "Benavente", "kalea", \Lambda, \Lambda, {"kenny", "Ray_Evans", "kenneth_ray", \Lambda, \Lambda, {"keith",
    "Santiago_Evans", "keith_santiago", \Lambda, \Lambda, {"ledecky10", "Ledecky", "katie", "2006-03-17",
    "2008-03-16"}, {"ledecky12", "Ledecky", "katie", "2008-03-17", "2010-03-16"}, {"ledecky14",
    "Ledecky", "katie", "2010-03-17", "2012-03-16"}, /* Roster added per requirements (all
    College Area Swim Team, LSC SI). */ /* The query is "First Last"; the match token is a
    lower-case substring */ /* that uniquely picks the CAST member among the search results —
    */ /* chosen to sidestep nicknames and same-name swimmers in other LSCs. */
{"bothwell-emma", "Emma_Bothwell", "emma", \Lambda, \Lambda, {"coombs-wally", "Wally_Coombs", "wally", \Lambda,
    \Lambda, {"cruz-eva", "Eva_Cruz", "cruz_rae", \Lambda, \Lambda, {"darrow-zoe", "Zoe_Darrow", "zoe", \Lambda, \Lambda,
    {"doan-eva", "Eva_Doan", "eva_doan", \Lambda, \Lambda, {"ellis-nova", "Nova_Ellis", "nova", \Lambda, \Lambda,
    /* Evans, Stella J is the same swimmer as "stella" above. */
{"fedyshyn-liliya", "Liliya_Fedyshyn", "liliya", \Lambda, \Lambda, {"glass-layla", "Layla_Glass", "layla",
    \Lambda, \Lambda, {"glass-logan", "Logan_Glass", "wiley", \Lambda, \Lambda, {"gosser-hannah", "Hannah_Gosser",
    "gosser", \Lambda, \Lambda, {"huynh-diamond", "Diamond_Huynh", "diamond", \Lambda, \Lambda, {"knuth-hannah",
    "Hannah_Knuth", "knuth", \Lambda, \Lambda, {"lefebvre-bailey-cleo", "Cleo_Lefebvre-Bailey", "cleo", \Lambda,
    \Lambda, {"mendez-giavanna", "Giavanna_Mendez", "mendez", \Lambda, \Lambda, {"mulvaney-sofia",
    "Sofia_Mulvaney", "sofia", \Lambda, \Lambda, {"shay-naia", "Naia_Shay", "naia", \Lambda, \Lambda, {"simmons-briar",
    "Briar_Simmons", "briar", \Lambda, \Lambda, {"simmons-presley", "Presley_Simmons", "aspen", \Lambda, \Lambda,
    {"soto-lucas", "Lucas_Soto", "erick", \Lambda, \Lambda, {"soto-vivienne", "Vivienne_Soto", "vivi", \Lambda,
    \Lambda, {"wittmershaus-addison", "Addison_Wittmershaus", "addison", \Lambda, \Lambda, {"zahner-elsie",
    "Elsie_Zahner", "elsie", \Lambda, \Lambda}};

#define NUM_SWIMMERS ((int)(sizeof SWIMMERS/sizeof SWIMMERS[0]))
```

See also sections 74, 75, 76, and 82.

This code is used in section 7.

74. *parse_opts_str* tokenises a comma-separated option string (the argument to *-o*) and sets bits in *g_opts*. Unknown tokens are silently ignored so that future options can be added without breaking existing invocations.

⟨Main function 73⟩ +≡

```
static void parse_opts_str(const char *s)
{
    /* Work on a writable copy so strtok can insert NUL bytes. */
    char *buf = strdup(s);
    if (!buf) return;
    char *tok = strtok(buf, ",");
    while (tok) {
        if (strcmp(tok, "fastest") == 0) g_opts |= OPT_FASTEST;
        else if (strcmp(tok, "csv") == 0) g_opts |= OPT_CSV;
        else if (strcmp(tok, "store") == 0) g_opts |= OPT_STORE;
        else if (strcmp(tok, "offline") == 0) g_opts |= OPT_OFFLINE;
        else if (g_nsel < MAX_SEL) g_sel[g_nsel++] = strdup(tok);
        tok = strtok(NULL, ",");
    }
    free(buf);
}
```

75. *parse_events_str* tokenises a comma-separated event-code string (the argument to *-e*) and appends each code to *g_events*. Codes that would overflow the *NUM_EVENTS*-entry array are silently dropped. The flag may be supplied multiple times; each invocation extends the list.

⟨Main function 73⟩ +≡

```
static void parse_events_str(const char *s)
{
    char *buf = strdup(s);
    if (!buf) return;
    char *tok = strtok(buf, ",");
    while (tok & g_nevents < NUM_EVENTS) {
        /* Trim leading spaces left by comma-split with spaces around commas. */
        while (*tok == ' ') tok++;
        strncpy(g_events[g_nevents], tok, sizeof g_events[0] - 1);
        g_events[g_nevents][sizeof g_events[0] - 1] = '\0';
        g_nevents++;
        tok = strtok(NULL, ",");
    }
    free(buf);
}
```

76. *print_usage* writes the full usage message to standard error. It is split into two sub-modules: one for the options section and one for the examples section.

⟨Main function 73⟩ +≡

```
static void print_usage(const char *prog){⟨Print usage options 77⟩⟨Print usage examples 81⟩}
```

77. The options section is split into three sub-modules so each *fprintf* stays inside the twenty-four line limit.

⟨Print usage options 77⟩ ≡

```
⟨Print usage flags 78⟩⟨Print usage swimmers 79⟩⟨Print usage event codes 80⟩
```

This code is used in section 76.

78. The `-o` keyword list. A token is either one of the four behaviour keywords below or a swimmer *id*; behaviour keywords and ids may be freely mixed. With no swimmer id every swimmer on the roster is processed.

⟨Print usage flags 78⟩ ≡

```
fprintf(stderr,
    "Usage: %s -o token[,token...] [-e event[,event...]]\n\n" " -o token,... comma-separated; each token is a behaviour keyword\n" " -r swimmer_id. Behaviour keywords: fastest print only the singl\
e fastest time per event\n" " -c csv emit CSV output (header + one line per time)\n" " -s store persist each row to\
Postgres swim-times via libpq\n" " -o offline read times from the\
Postgres swim-times DB\n" " -i instead of querying\
USA Swimming over the network\n" " -a Any other token selects a swimmer by\
id; with none, all are shown.\n\n", prog);
```

This code is used in section 77.

79. The swimmer ids are listed straight from the SWIMMERS roster so the help text can never drift out of sync with the table.

⟨Print usage swimmers 79⟩ ≡

```
fprintf(stderr, "swimmer ids (%d):\n", NUM_SWIMMERS);
for (int i = 0; i < NUM_SWIMMERS; i++) fprintf(stderr, "%s%-22s%s", (i % 3 == 0) ? "          " : "",
    SWIMMERS[i].id, (i % 3 == 2 || i == NUM_SWIMMERS - 1) ? "\n" : " ");
fputc('\n', stderr);
```

This code is used in section 77.

80. The `-e` event-code menu.

⟨Print usage event codes 80⟩ ≡

```
fprintf(stderr, " -e event,... restrict output to one or more event code\
s;\n" " -o comma-separated, or repeat -e for each ev\
ent.\n" " -s SCY codes: 50 FR SCY, 100 FR SCY, 200 FR SCY, 500 F\
R SCY,\n" " -f 1000 FR SCY, 1650 FR SCY,\n" " -l 50 FL SCY, 100 FL SCY, 50 BK SCY, 100 BK SCY,\n" " -b 50 BR SCY, 100 BR SCY, 100 IM SCY, 200 IM SCY\n" " -c LCM codes: 50\
FR LCM, 100 FR LCM, 200 FR LCM, 400 FR LCM,\n" " -f 800 FR LCM, 1500 FR LCM,\n" " -l 50 FL LCM, 100 FL LC\
M, 200 FL LCM,\n" " -b 50 BK LCM, 100 BK LC\
M, 200 BK LCM,\n" " -b 50 BR LCM, 100 BR LC\
M, 200 BR LCM,\n" " -c 200 IM LCM, 400 IM LCM\n\n");
```

This code is used in section 77.

81. The examples section illustrates common invocations.

⟨Print usage examples 81⟩ ≡

```
fprintf(stderr, "Examples:\n" " %s -o stella,fastest\n" " %s -o kalea,csv\n\
" " %s -o kenny -e \"100 FR SCY\" \n" " %s -o keith -e \"100 FR SCY, 50 FL SCY\
\" \n" " %s -o stella -e \"100 FR SCY\" -e \"50 FL SCY\" \n" " %s -o stella,csv,\
store #fetch + print + persist to DB\n" " %s -o kalea,offli\
ne #read from DB, no network\n" " %s -o ledecky10,fastest #Katie\
Leducky as a 9-10 yr old\n", prog, prog, prog, prog, prog, prog, prog, prog);
```

This code is used in section 76.

82. We initialise the global `libcurl` state, parse the optional `-o` and `-e` flags, then for each swimmer (subject to the selected swimmer ids in `g_sel`) resolve her `memberId` and iterate over events. If one or more `-e` codes were given only those events are fetched; otherwise all thirty-one are processed. In CSV mode a single header line is printed before the first data row. If no options at all are supplied, the usage message is printed and the program exits with status 1. The function body is split into three sub-modules.

⟨ Main function 73 ⟩ +≡

```
int main(int argc, char *argv[]) { ⟨ Parse command-line options 83 ⟩ ⟨ Check for empty invocation 84 ⟩ ⟨ Fetch
    and print swimmer times 85 ⟩ }
```

83. The option-parsing loop processes `-o` and `-e` flags via `getopt`.

⟨ Parse command-line options 83 ⟩ ≡

```
int ch;
while ((ch = getopt(argc, argv, "o:e:")) ≠ -1) {
    switch (ch) {
        case 'o': parse_opts_str(optarg);
            break;
        case 'e': parse_events_str(optarg);
            break;
        default: print_usage(argv[0]);
            return 2;
    }
}
```

This code is used in section 82.

84. If no options were supplied the usage message is printed and the program exits. Otherwise global curl state is initialised (skipped under `offline`, where no HTTP is performed), the CSV header is emitted if needed, and the DB pipe is opened if `store` was requested.

⟨ Check for empty invocation 84 ⟩ ≡

```
if (g_opts ≡ 0 ∧ g_nevents ≡ 0 ∧ g_nsel ≡ 0) {
    print_usage(argv[0]);
    return 1;
}
if (¬(g_opts & OPT_OFFLINE)) curl_global_init(CURL_GLOBAL_DEFAULT);
if (g_opts & OPT_CSV)
    printf("\nSwimmer\", \"Event\", \"Time\", \"Date\", \"Standard\", \"Meet\"\\n");
if (g_opts & (OPT_STORE | OPT_OFFLINE)) {
    int ok = db_open();
    if (¬ok ∧ (g_opts & OPT_OFFLINE)) {
        fputs("Error: _offline_mode_requires_a_DB_connection\\n", stderr);
        return 1;
    }
}
```

This code is used in section 82.

85. Each swimmer is visited in turn. The swimmer-filter mask is applied before the expensive *memberId* lookup. The DB pipe (if any) is flushed and closed before `libcurl` is torn down.

```

⟨Fetch and print swimmer times 85⟩ ≡
  for (int s = 0; s < NUM_SWIMMERS; s++) {⟨Process one swimmer 86⟩}
  db_close();
  if (¬(g_opts & OPT_OFFLINE)) curl_global_cleanup();
  return 0;

```

This code is used in section 82.

86. One swimmer is resolved and all requested events are fetched. Under `offline` the network *memberId* lookup is bypassed and the swimmer's display name is taken from the database in *offline_fetch*. The online branch is split into its own sub-chunk.

```

⟨Process one swimmer 86⟩ ≡
  if (g_nsel > 0) {
    int selected = 0;
    for (int k = 0; k < g_nsel; k++)
      if (strcmp(g_sel[k], SWIMMERS[s].id) == 0) {
        selected = 1;
        break;
      }
    if (¬selected) continue;
  }
  if (g_opts & OPT_OFFLINE) { ⟨Fetch events offline 89⟩continue; } ⟨Online swimmer dispatch 87⟩

```

This code is used in section 85.

87. The online path resolves a *memberId* and dispatches the per-event fetcher. A failed lookup is fatal; the DB pipe is closed and curl is torn down before exit.

```

⟨Online swimmer dispatch 87⟩ ≡
  const char *name = Λ;
  char *key = lookup_member_id(SWIMMERS[s].search_query, SWIMMERS[s].match_substr, &name);
  if (¬key) {
    db_close();
    curl_global_cleanup();
    return 1;
  }
  if (¬(g_opts & OPT_CSV))
    printf("Swimmer: %s %s (MemberId: %s)\n\n", name ? name : "(unknown)", key);
  ⟨Fetch events for swimmer 88⟩free(key);
  free((void *) name);
  if (¬(g_opts & OPT_CSV)) putchar('\n');

```

This code is used in section 86.

88. Either the user-requested events or all thirty-one default events are fetched online. The optional age window from the **Swimmer** record is forwarded to *fetch_times*.

⟨ Fetch events for swimmer **88** ⟩ ≡

```

if (g_nevents > 0) {
    for (int i = 0; i < g_nevents; i++)
        fetch_times(key, g_events[i], name, g_opts, SWIMMERS[s].date_min, SWIMMERS[s].date_max);
}
else {
    for (int i = 0; i < NUM_EVENTS; i++)
        fetch_times(key, EVENTS[i], name, g_opts, SWIMMERS[s].date_min, SWIMMERS[s].date_max);
}

```

This code is used in section **87**.

89. The offline counterpart calls *offline_fetch* once per event. No HTTP request is performed, no *memberId* is resolved, and the DB pipe is irrelevant (**store** composes only with online runs).

⟨ Fetch events offline **89** ⟩ ≡

```

if (g_nevents > 0) {
    for (int i = 0; i < g_nevents; i++) offline_fetch(&SWIMMERS[s], g_events[i], g_opts);
}
else {
    for (int i = 0; i < NUM_EVENTS; i++) offline_fetch(&SWIMMERS[s], EVENTS[i], g_opts);
}
if (¬(g_opts & OPT_CSV)) putchar(' \n ');

```

This code is used in section **86**.

90. Glossary.

The following terms and interfaces appear throughout this program.

91.

USA Swimming times API. The first-party REST service at `times-api.usaswimming.org` that backs the public data hub at `data.usaswimming.org`. It replaced the Sisense JAQL analytics API this program originally used (now decommissioned for public callers). Requests are ordinary HTTP GET/POST calls returning JSON; the OpenAPI description is published at `.../swagger/v1/swagger.json`.

Anonymous data-hub authentication. The times API uses no bearer token for public data. Every request instead carries three headers: `Usas-Sub-Id` (the caller's subject, or the literal `Anonymous` when not signed in), `AppName: DataHub`, and a `Device-Id`. The server validates the *format* of the `Device-Id`—base64 of "`<platform> - <vendor> - <fingerprint> - <millis>`" with the first five base64 characters repeated after the fifteenth—so *device.id* mints a conforming value at run time.

memberId. The service's identifier for a swimmer, an alphanumeric token such as `6CD35348E5824C` (it replaces the old numeric `PersonKey`).

92. USA Swimming REST API Calls. Both endpoints live under `https://times-api.usaswimming.org/swims/Times` and carry `Content-Type: application/json` plus the anonymous auth headers above.

- **Member search.** POST `/GetMembersForFilters` with body `{"name":"<query>"}`. Returns a JSON array of member objects, each with (among others) `memberId`, `fullName`, `clubName`, and `lscCode`. This program scans the array for the first member whose lower-cased `fullName` contains the match substring and keeps that record's `memberId`.
- **Best-times fetch.** POST `/BestTimes` with body `{"memberId":"<id>", "strokeAbbreviation":"<abbr>","distance"`. Returns a JSON array with one object per course (SCY and LCM) for that stroke-and-distance pair, each carrying `eventCode`, `swimTime` (formatted), `swimDate` ("Mon DD, YYYY"), `meetName`, and `timeStandard` (B, BB, A, AA, AAA, AAAA, or an elite label; JSON null when none was earned). This is the query the data-hub "Best Times" page issues, and unlike the older `GetBestTimesForMember` GET it exposes date, meet, and standard to anonymous callers. One POST is issued per distinct stroke-and-distance pair. The still richer per-swim endpoints (`GetAllTimesForFilters`, `GetSwimmerMeets`) require a signed-in subject and return 403 otherwise.

93. libcurl API Calls. This program uses the libcurl “easy” interface for synchronous HTTP. All functions return a *CURLcode* (zero = `CURLE_OK`) except where noted.

- `curl_global_init(flags)`.

Parameter: `flags` (long) — a bitmask of subsystems to initialise. This program passes `CURL_GLOBAL_DEFAULT`, which enables SSL and the Windows socket layer on that platform. Must be called once before any other libcurl function. Returns a *CURLcode*; this program ignores the return value because failure is treated as fatal by the subsequent easy calls.

- `curl_global_cleanup(void)`. Releases all resources allocated by `curl_global_init`. Must be called once after all easy handles have been cleaned up. Returns nothing.
- `curl_easy_init(void)`. Allocates and returns a new easy handle (a `CURL *`). Returns NULL on failure. Each call to `http_request` creates its own handle and destroys it before returning, so handles are never shared between requests.
- `curl_easy_setopt(handle, option, value)`.

Parameters:

- `handle` (`CURL *`): the easy handle.
- `option` (`CURLOption`): a constant selecting the behaviour to configure. Options used here:
`CURLOPT_URL` (`char *`) — the request URL.
`CURLOPT_HTTPHEADER` (`struct curl_slist *`) — linked list of extra HTTP headers (`Content-Type` and the anonymous `Usas-Sub-Id`, `AppName`, `Device-Id` headers).
`CURLOPT_POSTFIELDS` (`char *`) — the POST body; set only when a body is supplied, which also switches the method to POST (best-times fetches are plain GETs).
`CURLOPT_WRITEFUNCTION` (function pointer) — callback invoked for each response chunk; signature `size_t cb(void*,size_t,size_t,void*)`.
`CURLOPT_WRITEDATA` (`void *`) — the user-data pointer passed as the fourth argument to the write callback; here a pointer to the **Buffer** accumulator.
- `value`: type depends on `option` (see above).
- `curl_easy_perform(handle)`.

Parameter: `handle` (`CURL *`). Executes the configured request synchronously, invoking the write callback for each received chunk. Returns `CURLE_OK` on success or a non-zero error code; on failure `http_request` frees the partial buffer and returns NULL.

- `curl_easy_cleanup(handle)`.

Parameter: `handle` (`CURL *`). Releases all resources associated with the handle. The handle must not be used after this call.

- `curl_slist_append(list, string)`.

Parameters:

- `list` (`struct curl_slist *`): existing list head, or NULL to start a new list.
- `string` (`const char *`): the string to append. Returns the new list head, or NULL on allocation failure. Used to build the two-header list (`Content-Type` then `Authorization`).
- `curl_slist_free_all(list)`.

Parameter: `list` (`struct curl_slist *`). Frees every node in the linked list. Called immediately after `curl_easy_perform` so the headers are released before the handle.

94. POSIX System Calls and Library Functions. The following identifiers from the POSIX.1-2008 standard are used directly in this program. Each entry gives the C signature, a description of each parameter, and a note on how the program uses it.

- `int fprintf(FILE *stream, const char *fmt, ...)`.

Parameters:

- `stream`: destination file (`stderr` here).
- `fmt`: printf-style format string.
- `...`: values substituted into `fmt`. Writes formatted output to `stream`; returns the character count or a negative value on error. Used to report lookup and HTTP failures to standard error.

- `int fputs(const char *s, FILE *stream)`.

Parameters:

- `s`: null-terminated string to write.
- `stream`: destination file (`stderr` here). Writes `s` without a trailing newline; returns non-negative on success or EOF on error. Used for fixed error messages where no formatting is needed.

- `void free(void *ptr)`.

Parameter:

- `ptr`: pointer to a heap block, or NULL (in which case nothing happens). Releases the block back to the heap. Called on every heap string (response buffers, duplicated names and keys) when they are no longer needed.

- `void *memcpy(void *dst, const void *src, size_t n)`.

Parameters:

- `dst`: destination address.
- `src`: source address.
- `n`: number of bytes to copy. Copies exactly `n` bytes from `src` to `dst`; regions must not overlap. Returns `dst`. Used in *write_cb* to append each network chunk to the buffer, and in *scan_string* to copy a JSON string value.

- `int printf(const char *fmt, ...)`.

Parameters:

- `fmt`: printf-style format string.
- `...`: values substituted into `fmt`. Writes formatted output to standard output; returns the character count or a negative value on error. Used for all swimmer and time-table output.

- `int putchar(int c)`.

Parameter:

- `c`: character value (as `unsigned char` cast to `int`). Writes one character to standard output; returns the character written, or EOF on error. Used to emit a blank line (`'\n'`) after each event section.

- `void *realloc(void *ptr, size_t size)`.

Parameters:

- `ptr`: existing heap block, or NULL.
- `size`: new size in bytes. Returns a pointer to the resized block (possibly moved), or NULL if allocation fails (the original block is unchanged on failure). When `ptr` is NULL the call is equivalent to `malloc`. Used in *write_cb* to grow the response buffer incrementally as libcurl delivers each chunk.

- `int snprintf(char *buf, size_t n, const char *fmt, ...)`.

Parameters:

- `buf`: destination character array.
- `n`: maximum bytes to write, including the null terminator.
- `fmt`: printf-style format string.
- `...`: values substituted into `fmt`. Writes at most `n-1` formatted characters to `buf` and always null-terminates. Returns the number of characters that would have been written had the buffer been unlimited (so a return value $\geq n$ signals truncation). Used to assemble URL strings, JSON bodies, and the bearer-token header.

- `char *strchr(const char *s, int c)`.

Parameters:

- **s**: string to search.
- **c**: character to find (compared as `unsigned char`). Returns a pointer to the first occurrence of **c** in **s**, including the terminator if **c** is `'\0'`, or `NULL`. Used in *scan_string* to find the closing double-quote of a JSON string value.
- `char *strdup(const char *s)`.
Parameter:
 - **s**: null-terminated string to duplicate. Allocates a new heap block of `strlen(s)+1` bytes, copies **s** into it, and returns the pointer; returns `NULL` on failure. Used in *lookup_member_id* to persist the swimmer's full name and memberId string across the lifetime of a query.
- `size_t strlen(const char *s)`.
Parameter:
 - **s**: null-terminated string. Returns the number of bytes before the null terminator. Used to compute loop bounds when lower-casing names and to advance past a search needle in *scan_string*.
- `char *strncpy(char *dst, const char *src, size_t n)`.
Parameters:
 - **dst**: destination array (at least **n** bytes).
 - **src**: source string.
 - **n**: maximum bytes to copy. Copies up to **n** bytes; if **src** is shorter than **n**, the remainder of **dst** is zero-filled. If **src** is at least **n** bytes long, **dst** will *not* be null-terminated. Returns **dst**. This program always writes `dst[sizeof dst - 1] = '\0'` after the call to guarantee termination. Used to copy the swimmer name and requested event codes; fixed `TimeRow` fields are filled with *snprintf* instead.
- `char *strstr(const char *hay, const char *needle)`.
Parameters:
 - **hay**: string to search within.
 - **needle**: substring to search for. Returns a pointer to the first occurrence of **needle** in **hay**, or `NULL` if not found. The workhorse of the JSON scanner: used to locate key names (`"memberId"`, `"fullName"`, `"swimTime"`, ...) and to advance through the raw REST response text.
- `double strtod(const char *s, char **endptr)`.
Parameters:
 - **s**: string containing a floating-point number.
 - **endptr**: if non-`NULL`, receives a pointer to the first character not consumed by the conversion. Returns the parsed `double`; sets ***endptr** past the converted text. Used by *time_to_seconds* to convert each colon-separated group of a formatted time (e.g. `"1:40.36"`) to a `double` sort key.
- `int sscanf(const char *s, const char *fmt, ...)`.
Parameters:
 - **s**: input string to parse.
 - **fmt**: scanf-style conversion string.
 - **...**: pointers to receive the converted values. Reads formatted input from **s**; returns the number of input items successfully matched and assigned (which may be fewer than requested, or `EOF` on failure). Used by *normalize_date* to split `"Mon DD, YYYY"` into a month abbreviation, day, and year, and by *build_member_cache* to split an event code such as `"100 FR SCY"` into its numeric distance and stroke abbreviation.

95. Index.

Anonymous: [12](#).
AppName: [12](#).
argc: [82](#), [83](#).
argv: [82](#), [83](#), [84](#).
b: [23](#).
base64: [24](#), [25](#).
bi: [51](#).
body: [26](#), [27](#), [44](#), [45](#), [56](#).
buf: [27](#), [28](#), [51](#), [74](#), [75](#).
Buffer: [19](#), [23](#), [27](#), [93](#).
build_device_id: [12](#).
build_member_cache: [58](#), [60](#), [94](#).
B64: [24](#).
cache_add: [55](#), [57](#).
CacheEntry: [53](#), [55](#).
ch: [83](#).
code: [58](#).
CONNECTION_OK: [37](#).
curl: [27](#), [28](#).
CURL: [27](#).
curl_easy_cleanup: [28](#).
curl_easy_init: [27](#).
curl_easy_perform: [28](#), [93](#).
curl_easy_setopt: [27](#).
curl_global_cleanup: [85](#), [87](#).
CURL_GLOBAL_DEFAULT: [84](#).
curl_global_init: [84](#), [93](#).
curl_slist: [27](#).
curl_slist_append: [27](#).
curl_slist_free_all: [28](#).
CURLcode: [28](#), [93](#).
CURLE_OK: [28](#).
CURLOPT_HTTPHEADER: [27](#).
CURLOPT_POSTFIELDS: [27](#).
CURLOPT_URL: [27](#).
CURLOPT_WRITEDATA: [27](#).
CURLOPT_WRITEFUNCTION: [27](#).
data: [19](#), [23](#), [28](#).
DataHub: [12](#).
date: [20](#), [40](#), [55](#), [57](#), [63](#), [65](#), [71](#).
date_max: [21](#), [53](#), [60](#), [67](#), [71](#), [73](#), [88](#).
date_min: [21](#), [53](#), [60](#), [67](#), [71](#), [73](#), [88](#).
day: [54](#).
db_close: [36](#), [38](#), [85](#), [87](#).
db_conn: [36](#), [37](#), [38](#), [39](#), [40](#), [62](#), [67](#), [70](#).
db_conn_string: [35](#), [37](#).
db_insert_row: [40](#), [64](#).
DB_INSERT_STMT: [36](#), [39](#), [40](#).
db_open: [36](#), [37](#), [84](#).
dev_hdr: [27](#).
Device: [12](#), [24](#), [25](#).
device_id: [25](#), [27](#), [91](#).
dist: [56](#), [58](#).
done: [58](#), [59](#).
e: [31](#), [35](#), [55](#).
event: [40](#), [53](#), [55](#), [57](#), [61](#).
event_code: [60](#), [61](#), [63](#), [64](#), [65](#), [68](#), [69](#).
eventCode: [52](#), [57](#).
EVENTS: [10](#), [13](#), [58](#), [88](#), [89](#).
fetch_pair: [56](#), [58](#).
fetch_times: [60](#), [88](#).
flen: [47](#).
fprintf: [37](#), [39](#), [40](#), [48](#), [70](#), [78](#), [79](#), [80](#), [81](#).
fputc: [79](#).
fputs: [45](#), [70](#), [84](#).
free: [28](#), [48](#), [56](#), [74](#), [75](#), [87](#).
full_name: [47](#).
fullName: [47](#).
f0: [71](#).
f1: [71](#).
f2: [71](#).
f3: [71](#).
f4: [71](#).
f5: [71](#).
g_cache: [53](#), [55](#), [58](#), [61](#).
g_cache_member: [53](#), [58](#), [60](#).
g_cache_n: [53](#), [55](#), [58](#), [61](#).
g_events: [17](#), [58](#), [75](#), [88](#), [89](#).
g_nevents: [17](#), [58](#), [75](#), [84](#), [88](#), [89](#).
g_nsel: [17](#), [74](#), [84](#), [86](#).
g_opts: [17](#), [74](#), [84](#), [85](#), [86](#), [87](#), [88](#), [89](#).
g_sel: [17](#), [74](#), [82](#), [86](#).
GET: [26](#).
getenv: [35](#).
getopt: [83](#).
getpid: [25](#).
hdrs: [27](#), [28](#).
http_request: [26](#), [45](#), [56](#), [93](#).
i: [24](#), [47](#), [50](#), [58](#), [61](#), [63](#), [64](#), [65](#), [70](#), [79](#), [88](#), [89](#).
id: [17](#), [21](#), [25](#), [73](#), [79](#), [86](#).
Id: [12](#), [24](#), [25](#).
in: [24](#), [54](#).
insertion_sort: [50](#), [62](#).
j: [50](#), [59](#).
k: [86](#).
key: [31](#), [46](#), [47](#), [48](#), [58](#), [59](#), [87](#), [88](#).
len: [24](#), [31](#).
like_pat: [69](#).
lim: [62](#), [63](#), [64](#), [65](#).
lookup_member_id: [41](#), [42](#), [87](#), [94](#).
lower: [47](#).
M: [54](#).

- m*: [54](#).
- main*: [82](#).
- match_substr*: [21](#), [41](#), [42](#), [46](#), [47](#), [67](#), [69](#), [73](#), [87](#).
- max_len*: [31](#).
- MAX_SEL*: [17](#), [74](#).
- MAX_TIMES*: [11](#), [61](#), [70](#).
- meet*: [20](#), [40](#), [55](#), [57](#), [63](#), [65](#), [71](#).
- meetName*: [52](#), [57](#).
- member*: [47](#).
- member_id*: [56](#), [58](#), [60](#).
- memberId*: [2](#), [41](#), [46](#), [47](#), [48](#), [52](#), [82](#), [85](#), [86](#), [87](#), [89](#).
- memcpy*: [23](#), [31](#).
- mon*: [54](#).
- n*: [23](#), [25](#), [50](#), [54](#), [58](#).
- name*: [44](#), [46](#), [47](#), [48](#), [87](#), [88](#).
- nd*: [58](#), [59](#).
- needle*: [31](#).
- nmemb*: [23](#).
- normalize_date*: [54](#), [55](#), [94](#).
- now*: [25](#).
- nrows*: [61](#), [62](#), [65](#), [70](#), [71](#).
- null*: [57](#).
- NUM_EVENTS*: [11](#), [13](#), [17](#), [53](#), [55](#), [58](#), [59](#), [75](#), [88](#), [89](#).
- NUM_SWIMMERS*: [73](#), [79](#), [85](#).
- o*: [24](#).
- offline_fetch*: [21](#), [67](#), [68](#), [86](#), [89](#).
- ok*: [84](#).
- OPT_CSV*: [16](#), [62](#), [74](#), [84](#), [87](#), [89](#).
- OPT_FASTEST*: [16](#), [62](#), [65](#), [74](#).
- OPT_OFFLINE*: [16](#), [74](#), [84](#), [85](#), [86](#).
- OPT_STORE*: [16](#), [62](#), [74](#), [84](#).
- optarg*: [83](#).
- opts*: [60](#), [62](#), [65](#), [68](#).
- out*: [24](#), [31](#), [54](#).
- out_name*: [41](#), [42](#), [48](#).
- out_sz*: [24](#).
- p*: [23](#), [31](#), [46](#), [57](#).
- params*: [69](#), [70](#).
- parse_events_str*: [75](#), [83](#).
- parse_opts_str*: [74](#), [83](#).
- PGconn*: [36](#).
- PGRES_COMMAND_OK*: [39](#), [40](#).
- PGRES_TUPLES_OK*: [70](#).
- PGresult*: [39](#), [40](#), [70](#).
- pos*: [31](#), [54](#).
- POST*: [26](#).
- PQclear*: [39](#), [40](#), [70](#).
- PQconnectdb*: [37](#).
- PQerrorMessage*: [37](#), [39](#), [40](#), [70](#).
- PQexecParams*: [70](#).
- PQexecPrepared*: [40](#).
- PQfinish*: [37](#), [38](#), [39](#).
- PQgetvalue*: [71](#).
- PQntuples*: [70](#).
- PQprepare*: [39](#).
- PQresultStatus*: [39](#), [40](#), [70](#).
- PQstatus*: [37](#).
- print_usage*: [76](#), [83](#), [84](#).
- printf*: [63](#), [65](#), [84](#), [87](#).
- prog*: [76](#), [78](#), [81](#).
- ptr*: [23](#).
- putchar*: [65](#), [87](#), [89](#).
- q*: [69](#).
- r*: [40](#).
- raw*: [25](#).
- rc*: [28](#).
- realloc*: [23](#).
- rem*: [24](#).
- res*: [40](#), [70](#), [71](#).
- resp*: [45](#), [46](#), [48](#), [56](#), [57](#).
- row*: [53](#), [55](#), [61](#).
- rows*: [50](#), [61](#), [62](#), [63](#), [64](#), [65](#), [70](#), [71](#).
- s*: [51](#), [74](#), [75](#), [85](#).
- scan_string*: [31](#), [47](#), [57](#), [94](#).
- search_query*: [21](#), [41](#), [42](#), [44](#), [48](#), [73](#), [87](#).
- seen*: [59](#).
- selected*: [86](#).
- SFPersonSearchFilters*: [44](#).
- size*: [19](#), [23](#).
- sk*: [40](#).
- snprintf*: [25](#), [27](#), [31](#), [40](#), [43](#), [44](#), [54](#), [55](#), [56](#), [58](#), [59](#), [69](#), [94](#).
- sort_key*: [20](#), [40](#), [50](#), [55](#), [71](#).
- sscanf*: [54](#), [58](#).
- standard*: [20](#), [40](#), [55](#), [63](#), [65](#), [71](#).
- std*: [57](#).
- stderr*: [37](#), [39](#), [40](#), [45](#), [48](#), [70](#), [78](#), [79](#), [80](#), [81](#), [84](#).
- stime*: [57](#).
- strchr*: [31](#).
- strcmp*: [59](#), [60](#), [61](#), [71](#), [74](#), [86](#).
- strdup*: [47](#), [74](#), [75](#).
- strlen*: [24](#), [31](#), [47](#).
- strncpy*: [71](#), [75](#).
- stroke*: [56](#), [58](#).
- strstr*: [31](#), [47](#), [54](#).
- strtod*: [51](#), [71](#).
- strtok*: [74](#), [75](#).
- Sub*: [12](#).
- sw*: [68](#), [69](#), [71](#).
- SWIM_TIMES_PGCONNINFO*: [34](#), [35](#).
- swimDate*: [52](#), [57](#).
- Swimmer**: [17](#), [21](#), [67](#), [68](#), [73](#), [88](#).
- swimmer*: [40](#).
- swimmer_buf*: [70](#), [71](#).

swimmer_name: [60](#), [63](#), [64](#), [70](#).
SWIMMERS: [1](#), [16](#), [21](#), [73](#), [79](#), [86](#), [87](#), [88](#), [89](#).
swimTime: [52](#), [57](#).
t: [24](#), [51](#).
time: [20](#), [25](#), [40](#), [55](#), [63](#), [65](#), [71](#).
time_to_seconds: [20](#), [51](#), [55](#), [94](#).
TimeRow: [20](#), [40](#), [50](#), [53](#), [61](#), [67](#), [70](#).
TIMES_API: [10](#), [12](#), [43](#), [56](#).
timeStandard: [52](#), [57](#).
tmp: [50](#).
tok: [74](#), [75](#).
total: [51](#), [70](#).
ud: [23](#).
url: [26](#), [27](#), [43](#), [45](#), [56](#).
Usas: [12](#).
use_e: [58](#).
vals: [40](#).
write_cb: [23](#), [27](#), [94](#).
yr: [54](#).

〈Build offline query params 69〉 Used in section 68.
 〈Build person-search URL 43〉 Used in section 42.
 〈Build person-search body 44〉 Used in section 42.
 〈Check for empty invocation 84〉 Used in section 82.
 〈Close DB connection 38〉 Used in section 36.
 〈Copy one libpq row 71〉 Used in section 70.
 〈Data hub credentials 12〉 Used in section 10.
 〈Database pipe 34〉 Used in section 7.
 〈Define offline_fetch 68〉 Used in section 67.
 〈Emit rows as CSV 63〉 Used in section 62.
 〈Emit rows as table 65〉 Used in section 62.
 〈Emit rows to DB 64〉 Used in section 62.
 〈Event table 13〉 Used in section 10.
 〈Fetch and print swimmer times 85〉 Used in section 82.
 〈Fetch events for swimmer 88〉 Used in section 87.
 〈Fetch events offline 89〉 Used in section 86.
 〈Global constants 10〉 Used in section 7.
 〈HTTP utilities 23, 24, 25, 26〉 Used in section 7.
 〈Includes 9〉 Used in section 7.
 〈Initialize curl handle 27〉 Used in section 26.
 〈Insert one row 40〉 Used in section 34.
 〈Issue HTTP request and return 28〉 Used in section 26.
 〈Issue person-search request 45〉 Used in section 42.
 〈JSON scanner 31〉 Used in section 7.
 〈LCM events 15〉 Used in section 13.
 〈Main function 73, 74, 75, 76, 82〉 Used in section 7.
 〈Match one person row 47〉 Used in section 46.
 〈Numeric constants 11〉 Used in section 10.
 〈Offline fetch 67〉 Used in section 7.
 〈Online swimmer dispatch 87〉 Used in section 86.
 〈Open DB connection 37〉 Used in section 36.
 〈Open and close DB connection 36〉 Used in section 34.
 〈Option flags 16, 17〉 Used in section 7.
 〈Parse BestTimes records 57〉 Used in section 56.
 〈Parse command-line options 83〉 Used in section 82.
 〈Person lookup 42〉 Used in section 7.
 〈Prepare insert statement 39〉 Used in section 37.
 〈Print usage event codes 80〉 Used in section 77.
 〈Print usage examples 81〉 Used in section 76.
 〈Print usage flags 78〉 Used in section 77.
 〈Print usage options 77〉 Used in section 76.
 〈Print usage swimmers 79〉 Used in section 77.
 〈Process one swimmer 86〉 Used in section 85.
 〈Read offline rows from libpq 70〉 Used in section 68.
 〈Resolve connection string 35〉 Used in section 34.
 〈Return person-search result 48〉 Used in section 42.
 〈SCY events 14〉 Used in section 13.
 〈Scan person-search result 46〉 Used in section 42.
 〈Select cached time for event 61〉 Used in section 60.
 〈Skip pair if already fetched 59〉 Used in section 58.
 〈Sort and emit rows 62〉 Used in sections 60 and 68.
 〈Times fetch 50, 51, 53, 54, 55, 56, 58, 60〉 Used in section 7.

⟨ Type definitions [19](#), [20](#), [21](#) ⟩ Used in section [7](#).

SWIM-TIMES

	Section	Page
Swim Times	1	1
Includes and constants	8	3
Data structures	18	5
HTTP utilities	22	6
JSON scanner	29	9
Database connection	33	10
Person lookup	41	12
Times fetch	49	14
Offline fetch	66	19
Main program	72	21
Glossary	90	27
Index	95	31